

Blueprint: LLD practice with explainable feedback

Live: <https://blueprint-lld.onrender.com> (free hosting: the first visit after a quiet spell takes about a minute to wake the server).

Blueprint helps learners **practise Low-Level Design repeatedly** and understand where a design can improve:

choose a problem → design → submit → get feedback → review → try again.

- **Draw the design** on a UML canvas: classes, interfaces and enums with their members, typed relationships in proper UML notation, and requirement chips dropped onto the class that owns them. Live checks badge classes as you draw, and undo/redo is supported.
- **Walk it through:** click classes in call order to show how a requirement works. Each call is checked against the diagram (can this class actually reach that one?) and turned into a sequence diagram.
- **Explainable feedback:** 16 deterministic design rules plus an AI reviewer grounded on them (Claude or Groq, or an offline simulator clearly labelled *AI (sim)*). You get a weighted rubric score and findings tagged *Rule* or *AI* that point at your own classes and link back into the editor.
- **Take the curveball:** the interviewer's change requests, from a deck per problem or **aimed at your own design's weakest point of change**. The next report measures the **blast radius**: which classes were added, which existing ones had to change, which stayed untouched, and which abstractions paid off.
- **Keep improving:** every version is kept. You can compare versions, and track scores, weakest criteria and achievements. A timed interview mode counts down the problem's estimated time.

Reviewer quick tour (5 minutes)

1. Open <https://blueprint-lld.onrender.com>. It runs the real Groq reviewer. To run locally instead: `npm install && npm run dev`, then <http://localhost:5173> (no API key needed).
2. On the home page, click **See a sample report**. A worked Parking Lot design is submitted and evaluated in the background, and the report opens when it's ready.
3. On the report, switch the diagram between **Findings** and **Walkthroughs**, and open a finding's class link to jump into the editor.
4. In the curveball deck, click **Aim one at my design**, then **Take this curveball**. On the canvas, add a class that implements an existing interface and watch the banner's blast-radius counter. Submit, and the next report opens on **Change since v1**.
5. Open **Progress** for the score trend, criterion averages and achievements.

Design answers to the brief's five questions are in [docs/DESIGN.md §5](#).

Document	Where
Research note	docs/RESEARCH.md
Design note	docs/DESIGN.md (classes, interfaces, evaluation, trade-offs)
Iteration log	docs/ITERATIONS.md (critique → backlog per iteration)
AI usage	AI_USAGE.md
Original plan (historical, day 1)	docs/PLAN.md

Quick start

Requires **Node.js ≥ 22.13** (uses the built-in `node:sqlite` driver, so there is no native build).

```
npm install
npm run dev          # API on :3001 + web on :5173 (proxying /api)
```

Open <http://localhost:5173>. **No API key is needed.** Without one, the AI reviewer is an offline simulator, clearly labelled *AI (sim)* in the UI. To use a real model, set one key:

```
GROQ_API_KEY=gsk_... npm run dev      # Groq (default model openai/gpt-oss-120b)
ANTHROPIC_API_KEY=sk-ant-... npm run dev # Claude (default model claude-opus-5)
```

Keys are read from the environment only. Never commit them.

Production (single process)

```
npm run build      # web → apps/web/dist, API → apps/api/dist/server.mjs
npm start          # serves the API, the web app and the evaluation worker on :3001
```

or with Docker:

```
docker build -t blueprint-llm .
docker run -p 8080:8080 -v blueprint-data:/data -e GROQ_API_KEY=... blueprint-llm
```

Deploy (free: Render + Supabase)

The repo contains a `Dockerfile` and a Render Blueprint (`render.yaml`) for one web service (API, web app and evaluation worker in one process) on Render's **free** plan. The free plan has no persistent disk, so data lives in a free **Supabase** Postgres database. The app switches to Postgres whenever `DATABASE_URL` is set, and creates its tables on first start.

1. **Supabase:** create a project. In **Connect**, copy the **Session pooler** connection string (it works over IPv4, which Render needs), and put your database password in place of `[YOUR-PASSWORD]` .
2. **Render: New** → **Blueprint**, connect the GitHub repository, and choose `main` . When prompted, paste `GROQ_API_KEY` and `DATABASE_URL` . Every other setting comes from `render.yaml` .
3. **Apply.** The first build takes a few minutes. Then `https://<your-app>.onrender.com/api/health` should report `"aiReviewer": "groq:openai/gpt-oss-120b"` .

Free-plan caveats:

- The service sleeps after 15 minutes without traffic, and the next visit takes about a minute to wake it.
- A free Supabase project pauses after a week without activity. Resume it from the Supabase dashboard; no data is lost.
- Without `DATABASE_URL` , the app runs on a local SQLite file that is wiped on every restart.
- With a paid Render plan, you can use a persistent disk and SQLite instead: add a `disk` mounted at `/data` and remove `DATABASE_URL` .

Every push to `main` redeploys.

Tests

```
npm test      # shared (Mermaid, walkthroughs, design diff) + API (unit + HTTP) + web (editor state)
npm run typecheck
npm run e2e    # real-browser walkthrough: needs the app running (BASE_URL, default :5173)
```

- **Unit:** state machine (every illegal transition), attempt invariants, all rules on good and bad designs, **fairness** (two structurally different valid designs score alike, a weak one clearly lower), scoring caps, LLM output parsing and repair, hallucination guardrails, retry/timeout/cache decorators, the Groq adapter, walkthrough analysis, design diff, curveball targeting with model fallback, achievements, practice context, and editor undo/redo and rename propagation.
- **Integration (HTTP):** the whole loop (start → hint → submit → feedback → improve → compare → history). Also covers 400/404/409/422 cases, learner isolation, the AI outage → partial report → retry path, crash → retries → `failed` , recovery of jobs abandoned by a dead worker, aimed curveballs, and the sample report.

- **E2E (apps/web/e2e/run.mjs)**: drives Chromium through the full journey in light, dark and mobile layouts: drawing, undo, walkthroughs, submitting, curveballs, a timed interview, progress and the sample report. It runs axe accessibility audits on every screen, saves screenshots, and **fails on any console error, uncaught exception, 5xx, horizontal overflow, or serious or critical WCAG A/AA violation**.
- **CI (.github/workflows/ci.yml)** runs typecheck, all tests and the build, then the e2e walkthrough against the production server, on every PR.

Configuration

Variable	Default	Purpose
PORT	3001	HTTP port
DATABASE_URL	—	Postgres connection string (e.g. Supabase). When set, it is used instead of SQLite.
DATABASE_PATH	./data/blueprint.db	SQLite file, used when DATABASE_URL is not set
LLM_PROVIDER	auto	auto (Claude if its key is set, else Groq if its key is set, else simulator), anthropic , groq , simulated , none (rules only)
ANTHROPIC_API_KEY	—	Enables the Claude reviewer
GROQ_API_KEY	—	Enables the Groq reviewer
LLM_MODEL	per provider	claude-opus-5 (Anthropic) or openai/gpt-oss-120b (Groq)
LLM_EFFORT	medium	low / medium / high
LLM_TIMEOUT_MS / LLM_MAX_RETRIES	90000 / 2	Per-call timeout and retries (retryable errors only)
LLM_SIMULATED_LATENCY_MS	2500	Makes the offline reviewer feel real in demos
LLM_SIMULATED_FAILURE_RATE	0	e.g. 0.5 to demo the partial-report and retry path
WORKER_CONCURRENCY	2	Parallel evaluations

How it is built

apps/api	Fastify API + in-process evaluation worker (TypeScript)
domain/	Attempt, Submission, SubmissionLifecycle (state machine), ports
formats/	SubmissionParser registry: structured form, Mermaid class diagram
evaluation/	16 DesignRules, LLM reviewer (prompt, schema, guardrails), LLM decorators (timeout/retry/cache), pipeline, scoring, comparison
application/	PracticeService, EvaluationService, ProgressService, LintService, CurveballService, achievements
infrastructure/	SQLite or Postgres repositories + durable job queue, problem catalogue
apps/web	React + TanStack Query + Tailwind; workspace editor, report, compare, progress
packages/shared	Design IR, problem schema, DTOs, Mermaid, walkthrough analysis, design diff
problems/	Problem catalogue as validated JSON (rubric weights, points of change, hints, curveballs); problems/samples/ holds worked sample designs

Key decisions (details in the design note):

- **Structured submission.** It makes feedback checkable and rehearses what interviewers probe for. Mermaid import/export keeps it fast.
- **A rubric of properties, not a reference answer.** Rules never look for specific class names, so alternative designs aren't penalised.
- **Rules establish the facts and the AI supplies judgement.** The AI is grounded on rule findings, schema-constrained, stripped of hallucinated class names, and capped (at most +25 above the rule evidence; a critical finding caps the criterion at 50).
- **Failure-tolerant evaluation.** A durable queue with retries and crash recovery. If the AI is down the learner still gets a rule-based report, and can retry the AI review.

- **Extensible by addition.** New format → `SubmissionParser`. New approach → `Evaluator` / `DesignRule`. New provider → `LlmClient`. New problem → a JSON file.

Limitations

- There are no accounts. Each browser gets an anonymous learner id, so history doesn't follow you across devices.
- Storage is SQLite (a local file) or Postgres (`DATABASE_URL`), behind the same repository and queue interfaces. The Postgres queue claims jobs with `FOR UPDATE SKIP LOCKED` , so several instances could share it. The evaluation worker still runs inside the web process.
- The offline simulator's judgement is heuristic. Real qualitative feedback needs `GROQ_API_KEY` or `ANTHROPIC_API_KEY` . The Groq path is covered by tests against mocked HTTP. It has not yet run against the live API from the build environment, where outbound access to Groq was blocked.
- Only four problems (one with a worked sample), and no authoring UI (problems are JSON files).
- The blast radius of a curveball is measured and shown but not scored, because legitimate refactors also change classes.
- The editor is designed for desktop. It works on tablets and phones, including a drawer for the brief, but it's not optimised for them.
- AI findings are matched across versions by title, which is best-effort. Rule findings are matched exactly.

AI usage

This project was built with **Claude Code** as a pair-programmer: it proposed designs, wrote most of the code and tests, drove a real browser to screenshot every screen, and critiqued its own output between iterations (see `docs/ITERATIONS.md`). The decisions below are the ones where the AI's first suggestion was *not* the one that shipped, or where it had to be constrained.

1. How to score a design: similarity vs. properties

- **Suggested:** store a reference solution per problem and score submissions by similarity to it (class-name overlap plus an LLM "how close is this" judgement).
- **Decision: rejected.** LLD has many valid answers. Similarity scoring penalises a learner who picks a defensible alternative, which is exactly the skill we want to reward. Problems instead declare *properties*: core concepts with synonyms, **points of change** that should sit behind an abstraction, and an extension scenario. Rules check those properties, never specific class names.

2. Should the LLM produce the score?

- **Suggested:** send the design to the LLM and ask for a 0–100 score and feedback.
- **Decision: accepted in part, heavily constrained.** The LLM judges the qualitative parts, but it receives the deterministic findings as trusted facts, must answer in a JSON schema (structured outputs, then Zod validation plus one repair retry), and its references to classes that don't exist are dropped. It can lift a criterion at most +25 above the rule evidence, and a critical rule finding caps the criterion at 50. Reason: scores need to be repeatable and explainable, and an unconstrained LLM contradicted obvious facts ("all requirements are covered" when two weren't).

3. Infrastructure for slow or failing evaluation

- **Suggested:** Redis + BullMQ for the job queue, and WebSockets for status updates.
- **Decision: rejected for the MVP.** The brief says a monolith is fine and the focus is LLD. The job queue is a `JobQueue` interface with SQLite and Postgres implementations (atomic claim, retries with backoff, stale-lock recovery on boot), and the client polls with a server-suggested interval. This gives the same reliability properties for a prototype with zero extra infrastructure, and the interface keeps the swap cheap. What was kept from the suggestion is the important behaviour: when the AI fails, the learner still gets the rule-based report (`evaluated_partial`) and can retry.

4. Who decides where the curveball aims: the model or the code?

- **Suggested:** for the "aim a curveball at my design" feature, send the design to the LLM and let it invent a change request that stresses the design.
- **Decision: rejected as stated, kept for wording only.** A model choosing the target can't be tested, varies between runs, and might aim at something the scoring rules consider fine. The **target is chosen deterministically**: the first point of change with no abstraction, then an abstraction with no implementations, then the least-exercised seam. It uses the same keyword matching as the scoring rule, so the curveball and the report agree. The model only rewrites the change request in an interviewer's voice. Its JSON is validated, and any failure falls back to a template, so the feature works offline and is unit-tested for valid, invalid and failing model answers.

5. Trusting the AI's own UI and feedback: the screenshot critique loop

- **Suggested:** after writing the UI, declare it done because it typechecks and the API tests pass.
- **Decision: rejected.** Every iteration ends with a Playwright walkthrough in real Chromium that screenshots each screen (light, dark, mobile) and **fails on any console error, uncaught exception, 5xx or horizontal**

overflow. Reviewing those screenshots found bugs that tests didn't:

- the diagram rendered `park(v): Ticket as park(v) : : Ticket;`
- the mobile header overflowed by 20px;
- the offline reviewer called `ParkingSpot` "the orchestrator" because it counted inheritance edges;
- the offline reviewer awarded 100/100.

All were fixed in iteration 2, with tests where possible.

Decisions that came from the human, not the AI

- **From form to canvas.** The first versions used a form-based editor (tabs of fields). The human rejected it ("I don't want just a question-and-answer kind of thing") and asked for real diagramming. That redirected iteration 4 to the UML canvas, and later iterations to walkthroughs and curveballs.
- **A critique pass after every iteration.** The human required each iteration to start by criticising the previous one. That is why `docs/ITERATIONS.md` has a critique and backlog per iteration, and why several regressions (contrast, a hooks-order bug, over-generous scoring) were caught rather than shipped.
- **Evaluate against the brief before polishing further.** Asking for a review against the deliverables led to the fairness test, which exposed two scoring rules as too lenient (an empty trade-offs section scored 80, and a class with 8 responsibilities was only a "minor" issue). Both were recalibrated.
- **Free deployment on Render + Supabase.** The AI's first plan used a paid Render persistent disk for SQLite. The human asked for a free setup and proposed Supabase, which led to the Postgres storage adapter (same repository and queue ports, tested on both). Groq was added as a provider because it was the key available.

6. "Healthy" is not the same as "working": checking the analysis on the live site

- **Suggested:** treat the deployment as verified because `/api/health` reported the Groq reviewer and the browser walkthrough passed.
- **Decision: rejected.** A live check now submits a weak, a reasonable and an improved design through the API and fails unless the scores come out in that order, the AI review really ran (not the simulator), and the AI cites only classes that exist. Its first run showed that **every live review had silently fallen back to rules only**: Groq had retired the configured model, and the health endpoint only reports configuration. The reviewer moved to `openai/gpt-oss-120b`; the same check also caught a lenient god-class rule and a rate-limit retry that ignored `retry-after`.

Where AI was *not* used for judgement

- The rubric criteria and weights, the point-of-change definitions per problem, and the rule thresholds (e.g. god class > 6 responsibilities) are explicit, reviewable data and constants, not model output.
- Without an API key, the "AI" reviewer is an offline simulator and the UI labels it **AI (sim)** everywhere, so demo output is never presented as a model's judgement.